

Kubernetes, as a City

A PLAIN-ENGLISH WALKTHROUGH, START TO FINISH

PART 1 — THE BASIC STRUCTURE

CLUSTER —the whole city

The entire system. Everything below lives inside it. Nothing exists outside the city.

NODE —a building

One machine. The city has several buildings — if one goes down, the others keep standing.

POD —an apartment

The smallest thing Kubernetes manages. You can't have "half an apartment."

CONTAINER — the tenant living inside

POD —an apartment

Another apartment, same building.

CONTAINER — the tenant living inside

NODE —a building

A second building in the same city.

POD —an apartment

Smallest unit again — one apartment, one tenant.

CONTAINER — the tenant living inside

PART 2 — KEEPING THINGS RUNNING

DEPLOYMENT —the landlord's standing order

A rule you give the city's management: "Always keep 3 apartments filled with this exact type of tenant." You never manage apartments one by one — you set this standing order once, and the city keeps it true forever. If a tenant leaves or an apartment breaks, a replacement appears automatically.

Example: You tell the deployment "always keep 3 payment-service tenants running." If one crashes under heavy traffic, you don't lift a finger — a new one is created elsewhere in the city to bring the count back to 3.

SCALING —changing the standing order's number

You don't touch apartments yourself — you just change the number in the standing order, up or down, and the city's management handles the rest. Scale up when more customers arrive, scale down when things are quiet.

Example: A big sale sends traffic through the roof. Instead of manually setting up new servers, you change the standing order from 3 to 10 — Kubernetes fills 7 more apartments automatically, and the directory (service) starts routing customers to all 10 right away.

SERVICE —the city directory / reception desk

Apartments get torn down and rebuilt constantly, and each new one has a different address. A Service is a single, fixed listing — "ask here for the payment service" — that always points to whichever apartment currently has a working tenant, no matter how many times it's changed behind the scenes.

Example: The website never needs to know an exact apartment number for the payment service. It just asks the directory for "payment service," and gets routed to whichever pod is alive right now.

ClusterIP — the internal-only intercom

Default option. Only works for people already inside the city. Nobody outside the city walls can reach it at all. Use this for things that should never be public — like a database.

NodePort — a side door, same number, on every building

Opens one specific door number on every building, so outsiders can walk up and get routed in. Works, but rough and exposed — mainly used for quick testing, not real production traffic.

Ingress — the city's proper front gate, with a receptionist

One smart gate at the edge of the city. Visitors say where they want to go, by name, and the gate routes them correctly. One gate can send different visitors to many different destinations — this is the real, production-grade entrance.

- Cluster (city) → Node (building) → Pod (apartment) → Container (tenant)
 - Deployment keeps the right number of apartments filled
 - Scaling changes that number, up or down
 - Service is the directory that always finds the current tenant
 - The exposure type decides who's allowed to use that directory

Check the cluster's nodes

NODE — BUILDING

```
kubectl get nodes
```

Lists all the **buildings (nodes)** in the **city (cluster)**, and whether each one is healthy (Ready).

Create a Deployment

DEPLOYMENT — STANDING ORDER

```
kubectl create deployment nginx-deployment --image=nginx --replicas=3
```

nginx-deployment— the name you're giving this standing order
--image=nginx — which tenant type to move in (the nginx app)
--replicas=3 — how many apartments to keep filled

Sets the **standing order (deployment)** — tells Kubernetes to always keep 3 **apartments (pods)** filled with an nginx **tenant (container)**.

Check your pods

POD — APARTMENT

```
kubectl get pods
```

```
kubectl get pods -o wide
```

-o wide — show extra detail, including which building (node) each apartment is in

Shows every **apartment (pod)** and its status. The "-o wide" version also shows which **building (node)** each apartment is in.

Check your deployment

DEPLOYMENT — STANDING ORDER

```
kubectl get deployment
```

Shows whether the **standing order (deployment)** is being met — e.g. "3/3" means all 3 desired **apartments (pods)** are up and occupied.

Scale a deployment

SCALING — ORDER NUMBER

```
kubectl scale deployment nginx-deployment --replicas=5
```

--replicas=5 — the new number in the standing order (was 3, now 5)

Changes the **standing order (deployment)** from 3 to 5 — Kubernetes automatically fills 2 more **apartments (pods)** to match.

Create a Service

SERVICE — DIRECTORY

```
kubectl expose deployment nginx-deployment --type=ClusterIP --port=80 --name=nginx-service
```

```
kubectl expose deployment nginx-deployment --type=NodePort --port=80 --name=nginx-service
```

--type=ClusterIP— internal-only intercom (nobody outside the city can use it)
--type=NodePort — same directory, but with a side door opened on every building
--port=80 — the "line" people dial to reach this directory listing
--name=nginx-service— the name shown on the directory listing

Creates the **directory listing (service)**. ClusterIP = internal-only intercom. NodePort = same directory, but with a side door opened on every **building (node)** so it's reachable from outside.

Check your service SERVICE — DIRECTORY

```
kubectl get service
```

Shows the **directory listing (service)** itself — for NodePort, also shows which door number (port) to use from outside.

Debugging a pod POD — APARTMENT

```
kubectl describe pod <pod-name>
```

<pod-name> — swap this for the actual apartment's name, from "kubectl get pods"

Gives the full story on one **apartment (pod)** — useful when something's wrong and you need to know why.

Debugging a container CONTAINER — TENANT

```
kubectl logs <pod-name>
```

<pod-name> — same idea, the apartment whose tenant you want to check on

Shows the actual output/errors from the **tenant (container)** living inside that apartment — like checking what they've actually been doing.

CONFIGMAP & SECRET —INSTRUCTIONS AND KEYS, KEPT SEPARATE FROM THE TENANT

Instead of hardcoding settings into a tenant's own belongings, you hand them a separate notice with instructions (ConfigMap) or a locked box with sensitive keys (Secret) — so the same tenant type can behave differently depending on what's handed to them, without changing the tenant itself.

Example: A database password is stored as a Secret and handed to the pod at startup — never written directly into the application code.

NAMESPACE —SPLITTING THE CITY INTO SEPARATE DISTRICTS

One cluster can be divided into isolated districts — commonly one each for dev, test, and prod — so resources in one district don't accidentally interfere with another, even though they're all part of the same city.

Example: A "test" namespace and a "prod" namespace can both have a pod named "web-app" without conflict — they're in different districts.

PERSISTENT VOLUME —STORAGE THAT SURVIVES THE TENANT MOVING OUT

Apartments are temporary — anything stored inside is normally lost when a pod is replaced. A Persistent Volume is storage that exists independently of any one apartment, so data survives even when the tenant changes.

Example: A database pod is replaced after a crash, but its actual data survives untouched, because it lived on a persistent volume, not inside the pod itself.

HEALTH CHECKS & ROLLING UPDATES —CHECKING A TENANT IS ACTUALLY OKAY, AND UPDATING WITHOUT A BLACKOUT

Kubernetes can regularly check whether a tenant is genuinely working, not just "present" — and when updating to a new version, it replaces pods a few at a time instead of all at once, so the city never goes fully dark during an update.

Example: An unresponsive tenant fails its health check and gets replaced automatically. A version update rolls out one pod at a time, so customers never see downtime.

Create a ConfigMap and a Secret**CONFIGMAP & SECRET**

```
kubectl create configmap app-config --from-literal=MODE=production
```

```
kubectl create secret generic db-password --from-literal=PASSWORD=hunter2
```

Stores non-sensitive settings in a **ConfigMap**, and sensitive values in a **Secret** — both kept separate from the application code itself.

Work with namespaces**NAMESPACE — DISTRICT**

```
kubectl create namespace test
```

```
kubectl get pods --namespace=test
```

Creates a new **district (namespace)**, then checks pods only within that specific district instead of the whole city.

Check persistent storage**PERSISTENT VOLUME**

```
kubectl get pv
```

```
kubectl get pvc
```

Lists available **persistent volumes** (the actual storage) and **persistent volume claims** (a pod's request to use some of that storage).

deployment.yaml, with a ConfigMap, Secret, and health check **ALL TOGETHER**

```
spec:
  containers:
  - name: web
    image: myapp
    envFrom:
    - configMapRef:
        name: app-config
    - secretRef:
        name: db-password
    livenessProbe:
      httpGet:
        path: /health
        port: 80
        periodSeconds: 10
    strategy:
      rollingUpdate:
        maxUnavailable: 1
```

- envFrom: configMapRef**— pulls in the ConfigMap's settings automatically
- secretRef** — pulls in the Secret's values automatically
- livenessProbe** — checks /health every 10 seconds; unhealthy pods get replaced
- maxUnavailable: 1**— during an update, at most 1 pod is down at a time

This is genuinely close to a real production deployment — configuration and secrets kept separate, health actively monitored, and updates rolled out safely rather than all at once.

deployment.yaml THE WRITTEN-DOWN STANDING ORDER

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx
          ports:
            - containerPort: 80

```

kind: Deployment— this file is a standing order
name: nginx-deployment— the name of this standing order
replicas: 3 — how many apartments (pods) to keep filled
containers: image: nginx— the tenant type to place inside each apartment
containerPort: 80— the door number the tenant answers on, inside the apartment

This is the same standing order from earlier, just written down as a file instead of typed as a command. Save it and run **kubectl apply -f deployment.yaml** — this is how real teams actually manage Kubernetes, since the file can be reviewed and stored in Git.

PART 6 — FOLLOWING ONE REQUEST THROUGH THE CITY

A customer visits the website → hits the **front gate (Ingress)** → the gate reads the address and routes them to the right **directory (Service)** → the directory picks a currently healthy **apartment (pod)** → the request reaches the **tenant (container)** inside, who does the actual work and sends a reply back the same way.

If that apartment's tenant is busy or the apartment fails mid-request, the directory simply routes the next request to a different apartment — the customer never notices anything changed behind the scenes.

PART 7 — QUICK SELF-TEST

1. What's the difference between a pod and a deployment?
2. If a node goes down, what happens to the pods on it?
3. Why can't a service just point to one fixed pod address?
4. Which exposure type would you use for a database, and why?
5. What does changing "replicas" actually do behind the scenes?
6. What's the difference between running a command directly vs. using a YAML file?

PART 8 — IF THE MANAGER ASKS...

"What's the difference between a pod and a container?"

A pod is the apartment — the unit Kubernetes manages. A container is the tenant living inside it, actually doing the work. Most pods hold exactly one container.

"What happens if a pod crashes?"

The deployment notices the replica count dropped below what it should be, and automatically creates a new pod elsewhere in the cluster to bring it back — no manual action needed.

"Why use a Service instead of the pod's own address?"

Pod addresses change every time a pod is replaced. A Service gives a single, stable address that always routes to whichever pods are currently healthy, so nothing else needs to track individual pod addresses.